

Anomaly Detection Of Audio Based On Real Audio Using GANs Model

A RESEARCH PROJECT REPORT

Submitted by

G. KAVYAKRUTHIKA NEERAJA HANNALA J. KALPANA

22RH1A6671

22RH1A6672

22RH1A6673

Under the Esteemed Guidance of

Dr. M. VAMSIKRISHNA

Associate professor

In partial fulfilment of the academic requirements for the Degree of

BACHELOR OF TECHNOLOGY

CSE-AI & ML



MALLA REDDY ENGINEERING COLLEGE FOR WOMEN

(Autonomous Institution-UGC, Govt. of India)

Accredited by NAAC with 'A+' Grade

**National Ranking by NIRF Innovation – Rank band (151-300), MHRD, Govt. of India
Approved by AICTE, Permanently Affiliated to JNTUH, ISO 9001:2015 Certified Institution**

Maisammaguda, Dhulapally, Secunderabad, Kompally-500100.

2025-26



MALLA REDDY ENGINEERING COLLEGE FOR WOMEN

(Autonomous Institution-UGC, Govt. of India)

Accredited by NAAC with 'A+' Grade

**National Ranking by NIRF Innovation – Rank band (151-300), MHRD, Govt. of India
Approved by AICTE, Permanently Affiliated to JNTUH, ISO 9001:2015 Certified Institution**

Maisammaguda, Dhulapally, Secunderabad, Kompally-500100.

DEPARTMENT OF CSE - AI & ML

CERTIFICATE

This is to certify that the Research project work entitled **Anomaly Detection Of Audio Based On Real Audio Using GANs Model** is carried out by [G.kavyaKruthika \(22RH1A6671\)](#), [H. Neeraja \(22RH1A6672\)](#), [J. Kalpana \(22RH1A6673\)](#), in partial fulfillment for the award of degree of BACHELOR OF TECHNOLOGY in CSE (AI & ML), Jawaharlal Nehru Technological University, Hyderabad during the academic year 2025- 2026.

Supervisor's Signature
Dr. M. VAMSIKRISHNA
Associate professor

Head of the Department
Dr. M. NARENDHAR
Professor

External Examiner



MALLA REDDY ENGINEERING COLLEGE FOR WOMEN

(Autonomous Institution-UGC, Govt. of India)

Accredited by NAAC with 'A+' Grade

National Ranking by NIRF Innovation – Rank band (151-300), MHRD, Govt. of India
Approved by AICTE, Permanently Affiliated to JNTUH, ISO 9001:2015 Certified Institution
Maisammaguda, Dhulapally, Secunderabad, Kompally-500100.

Department of CSE –AI & ML

DECLARATION

We hereby declare that Research project entitled **Anomaly Detection Of Audio Based On Real Audio Using GANs Model** submitted to Malla Reddy Engineering College For Women affiliated to Jawaharlal Nehru Technological University, Hyderabad(JNTUH) for the award of the Degree of Bachelor of Technology in CSE(AI & ML) is a result of original research work done by us. It is further declared that the Research Project report or any part thereof has not been previously submitted to any University or Institute for the award of Degree.

G.KavyaKruthika(22RH1A6671)

Hannala Neeraja (22RH1A6672)

J. Kalpana (22RH1A6673)

ACKNOWLEDGEMENT

We feel ourselves honored and privileged to place our warm salutation to our college **Malla Reddy Engineering College for Women** and **Department of CSE-AI&ML**. which gave us the opportunity to have expertise in engineering and profound technical knowledge.

We would like to deeply thank our honorable MLA, **Sri.Ch. Malla Reddy Garu**, founder chairman MRGI, the largest cluster of institutions in the state of Telangana for providing us with all the resources in the college to make our project Success.

We wish to convey gratitude to our **Principal Dr. Y. Madhavee Latha**, for providing us with the environment and mean to enrich our skills and motivating us in our endeavor and helping us to realize our full potential.

We would like to thank **Prof. A. Radha Rani**, Director of Computer Science and Engineering & Information Technology for encouraging us to take up a project on this subject and motivating us towards the Project Work.

We express our sincere gratitude to **Dr. M. Narendhar**, Head of the Department of CSE- AI&ML for inspiring us to take up a project on this subject and successfully guiding us towards its completion,

We would like to thank our guide **Dr. M. Vamsi Krishna** Associate Professor and all the Faculty members for their valuable guidance and encouragement towards the completion of our project work.

With Regards and Gratitude

G. KavyaKruthika (22RH1A6671)

HannalaNeeraja (22RH1A6672)

J. Kalpana (22RH1A6673)

ABSTRACT

The rapid advancement of artificial intelligence and voice synthesis technologies has led to the emergence of deepfake audio synthetically generated speech that closely mimics human voices. These audio forgeries pose significant threats in domains such as cybersecurity, media integrity, and digital forensics, as they can be misused for impersonation, misinformation, and fraud. Identifying such manipulated content is increasingly challenging due to the high-quality output of modern neural text-to-speech models like Tacotron2 and Wave Net. This project presents an AI-powered deepfake audio detection system using spectrogram analysis and deep learning. The core idea is to convert audio signals into Mel-spectrogram images, which effectively capture both temporal and frequency information. These images are then fed into a Convolutional Neural Network (CNN) that classifies the audio as either “Real” or “Fake.” The CNN model is trained on a diverse dataset containing both genuine and AI-generated speech samples. We implemented the system using Python with libraries like Librosa for audio processing, Matplotlib for visualization, and TensorFlow/Keras for model training. A simple GUI built with Tkinter allows users to upload audio files and view prediction results with confidence scores.

INDEX

TITLE	i
CERTIFICATE	ii
DECLARATION	iii
ACKNOWLEDGMENT	iv
ABSTRACT	v
1. INTRODUCTION	6
2. LITERATURE SURVEY	7-9
3. SOFTWARE REQUIREMENT ANALYSIS	10-18
3.1 Existing System	10
3.2 Proposed System	11
3.3 SDLC	12-15
3.4 System Requirements	16-18
4. SOFTWARE DESIGN	19-22
4.1 UML Diagrams	19
4.2 Class Diagram	20
4.3 Sequence Diagram	21
4.4 Deployment Diagram	22
5. IMPLEMENTATION	23-26
5.1 Python	23-24
5.2 Algorithms	25-26
6. SOURCE CODE	27-29
7. TESTING	30-32
8. RESULTS	33-36
9. CONCLUSION	37
10. FUTURE SCOPE	38
REFERENCES	39

CHAPTER 1

INTRODUCTION

With the rapid advancement of artificial intelligence and speech synthesis technologies, the generation of deepfake audio has become increasingly common. Deepfake audio refers to synthetically generated speech that closely mimics real human voices, often produced using advanced neural text-to-speech (TTS) models like Tacotron2, WaveNet, and voice cloning algorithms.

Deepfake audio can be used for malicious purposes such as impersonating individuals, bypassing voice authentication systems, spreading misinformation, or committing fraud. As synthetic speech becomes more realistic, it becomes increasingly difficult for humans and traditional detection tools to identify whether an audio clip is real or fake.

To address this issue, our project proposes an AI-based detection system that uses spectrogram analysis and deep learning to identify deepfake audio. The system converts audio inputs into Mel-spectrograms, which are visual representations of sound frequency over time. These spectrograms highlight subtle differences in pitch, tone, and noise artifacts that are often present in synthetic speech. A Convolutional Neural Network (CNN) is then trained to classify the spectrograms as either “Real” or “Fake.”

The model is developed using Python libraries such as Librosa, TensorFlow, and Matplotlib, and it is trained on datasets containing both genuine and fake voice samples. A user-friendly graphical interface (GUI) built with Tkinter allows users to test audio files easily.

This project provides a scalable and effective solution to detect deepfake audio and can be integrated into various applications such as voice-based authentication systems, digital media verification, and forensic audio analysis.

CHAPTER 2

LITERATURE SURVEY

Scene understanding—A survey

The rise of deepfake audio has become a significant concern in fields such as media security, digital forensics, cybersecurity, and online communication. Deepfake audio refers to voice recordings that are artificially generated or manipulated using advanced AI techniques like Generative Adversarial Networks (GANs) and neural text-to-speech models. These systems can replicate a person's voice with high accuracy, including tone, pitch, and speaking style. While these technologies have positive applications in virtual assistants, film dubbing, and accessibility tools, they also pose major risks.

Deepfake audio can be used for harmful purposes such as impersonation, voice spoofing, and social engineering attacks. Cybercriminals can exploit these fake voices to commit financial fraud, deceive individuals, bypass voice-based authentication systems, or spread false information. The ease of creating realistic deepfake audio with minimal technical expertise makes the threat more widespread.

Due to these risks, there has been a growing focus on developing automated and intelligent detection systems that can differentiate between real and fake audio. Identifying such manipulations is crucial to preserving the authenticity of voice data and maintaining trust in digital communication platforms.

Traditional Audio Spoof Detection Techniques

Earlier studies in spoofing detection primarily relied on manually extracted acoustic features such as pitch, jitter, shimmer, and Mel-Frequency Cepstral Coefficients (MFCCs). These handcrafted features were used with classifiers like Gaussian Mixture Models (GMMs), Support Vector Machines (SVMs), and Random Forests. Although such methods demonstrated acceptable performance in basic classification tasks, their ability to generalize to newer, more realistic AI-generated voices was limited. The **ASVspoof Challenge**, initiated in 2015 and continued in subsequent years (2017, 2019, 2021), served as a benchmark for evaluating spoofing detection methods. Early submissions to ASVspoof relied on MFCC-GMM pipelines, but by 2019, more complex systems using Constant Q Cepstral Coefficients (CQCC), phase features, and i-vectors were adopted. However, the rising sophistication of neural vocoders rendered many of these systems ineffective.

Deep Learning Approaches

To overcome the limitations of traditional feature extraction, researchers began exploring deep learning, particularly Convolutional Neural Networks (CNNs). CNNs learn hierarchical feature representations from spectrograms—visual representations of audio signals—and perform exceptionally well in classification tasks. A study by Albadawy et al. (2020) showed that deepfake audio could be effectively detected using a CNN trained on Mel-spectrograms, achieving over 90% accuracy.

Spectrograms enable the visualization of frequency variations over time, which helps CNN models identify synthetic artifacts often embedded in deepfake audio. Unlike humans who perceive audio linearly, CNNs can pick up high-frequency noise and discontinuities that are typically overlooked by conventional models.

In addition, Recurrent Neural Networks (RNNs), particularly Long Short-Term Memory (LSTM) networks, have been utilized to capture temporal dynamics in speech. While CNNs focus on local spatial features, RNNs model long-term dependencies and transitions in the audio signal, making them suitable for tasks such as voice spoof detection and speaker verification. However, LSTMs require significant computational resources and large amounts of training data.

Hybrid Deep Learning Models

Hybrid models that combine CNNs and RNNs have emerged as a more robust solution. In particular, CNN-BiLSTM architectures allow for both spatial feature extraction (via CNN) and temporal sequence modeling (via Bidirectional LSTM). These models have shown improved generalization and robustness across diverse datasets. For instance, in their work on spoofed audio detection, Müller et al. (2021) demonstrated that CNN-BiLSTM models achieved superior accuracy and lower false positives compared to standalone CNNs or RNNs. Furthermore, attention mechanisms are being introduced to focus the model on more informative regions of the spectrogram, enhancing interpretability and performance.

Transfer Learning and Pre-trained Models

Recent literature also explores **transfer learning**, wherein models like **ResNet**, **EfficientNet**, and **VGG-19**—originally trained on large image datasets—are fine-tuned on spectrogram data. These pre-trained architectures reduce the need for large labeled datasets and accelerate convergence.

Datasets for Deepfake Audio Detection

A variety of public datasets have been used in the development and evaluation of deepfake audio detectors. These include:

- ASVspoof 2019 & 2021: Benchmark datasets containing genuine and spoofed speech from multiple attack types including TTS and voice conversion.
- FakeYou Dataset: Contains user-generated deepfake voice clips using various celebrity voices.
- WaveFake Dataset: Focused on generative vocoders and includes challenging synthetic samples.
- Libri Speech and VoxCeleb: Commonly used for collecting real speech samples for training and validation.

The availability of such datasets has accelerated the development of robust detection models and allowed for standard benchmarking.

Challenges and Research Gaps

Despite advances, current systems still face challenges. Many models struggle to generalize across unseen deepfake generation techniques or to handle adversarial attacks. Environmental noise, varied recording conditions, and multilingual speech pose additional hurdles. Furthermore, deepfake detection must be efficient and lightweight enough for real-time deployment on edge devices or browsers.

Another critical concern is the explainability of AI models. Most deep learning models function as black boxes, which raises trust and accountability issues in security-critical applications. Efforts are ongoing to integrate explainable AI (XAI) into audio forensics for greater transparency.

Conclusion of Literature Survey

From traditional signal-based techniques to state-of-the-art deep learning architectures, deepfake audio detection has evolved rapidly in response to emerging threats. Spectrogram-based CNN models have proven particularly effective due to their ability to capture fine-grained patterns in frequency and time. Hybrid approaches, self-supervised learning, and transfer learning further enhance detection capabilities. However, generalization, explainability, and real-time performance remain open challenges. The proposed project builds upon these insights by implementing a CNN-based model trained on Mel-spectrograms to classify deepfake audio. With a user-friendly interface and high detection accuracy, it contributes to ongoing efforts in securing digital communication and preserving trust in digital communication platforms.

CHAPTER 3

SOFTWARE REQUIREMENT ANALYSIS

3. SYSTEM ANALYSIS

3.1 Existing System:

Current methods of detecting audio forgeries rely on manual analysis or simple feature extraction techniques such as MFCCs or audio fingerprints. These approaches often fail when adversarial attacks are introduced or when deepfake audio is generated with high-quality neural TTS models like Tacotron2 or WaveNet.

Disadvantages

1. Poor generalization across datasets
2. Inability to handle modern neural deepfakes

3.2 Proposed System:

The proposed model utilizes a hybrid deep learning architecture that processes audio by converting it into spectrogram images, followed by classification using Convolutional Neural Networks (CNN). It can distinguish between real and fake audio samples with high precision by leveraging spatial and frequency features embedded in the spectrograms. By transforming audio into spectrograms, the problem becomes an image classification task, enabling the model to detect subtle patterns often missed in raw waveforms.

The system consists of audio preprocessing, spectrogram generation, and a CNN-based classifier. Mel-spectrograms are generated using Librosa, resized, and normalized before being passed through the model. The CNN includes convolutional layers for feature extraction, pooling for dimensionality reduction, and dense layers for final classification. A simple GUI built with Tkinter allows users to upload audio files and receive predictions instantly.

This architecture is designed for high accuracy, scalability, and ease of integration into real-world applications such as media platforms, security systems, and forensic tools.

Advantages

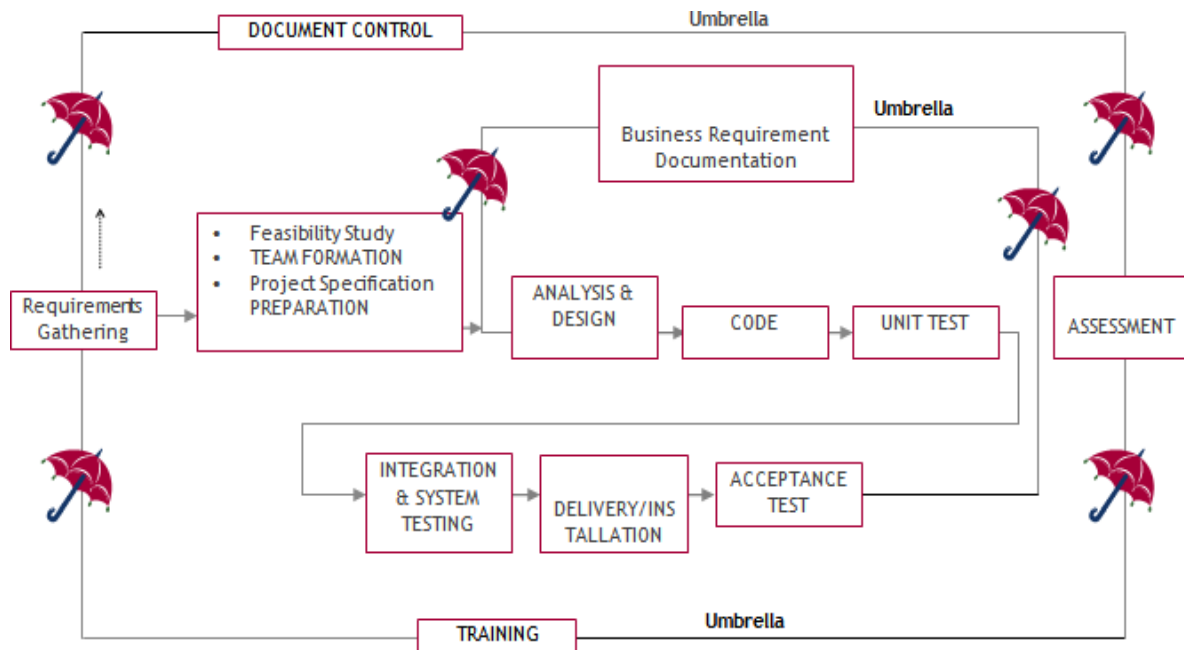
1. High detection accuracy
2. End-to-end automation
3. Spectrogram-based feature analysis
4. Easy-to-use GUI
5. Scalable for real-world use
6. Robust across various audio types

MODULE DESCRIPTION:

1. **Audio Upload Module** – User uploads .wav file
2. **Spectrogram Generator** – Converts audio to Mel spectrogram
3. **Feature Extractor** – Normalizes input for CNN
4. **Classifier Model** – Predicts real or fake audio
5. **Prediction Module** – Displays result with confidence
6. **Graphical UI** – Tkinter-based user interface for interaction

3.3._PROCESS MODEL USED WITH JUSTIFICATION

SDLC (Umbrella Model):



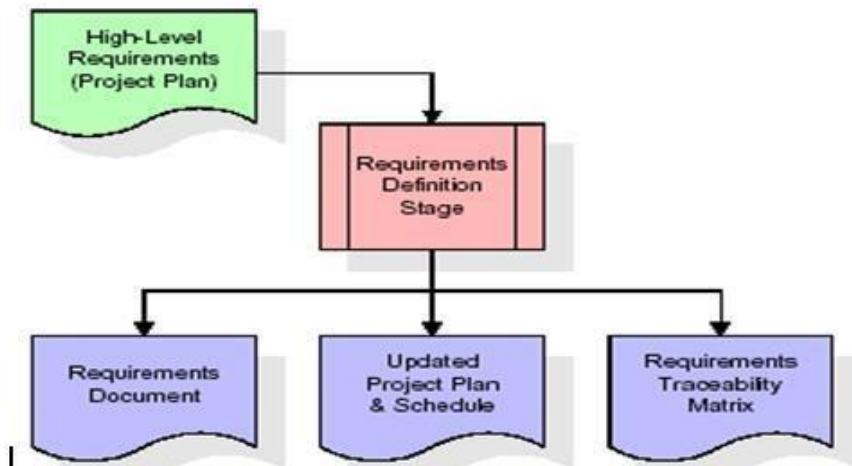
SDLC is nothing but Software Development Life Cycle. It is a standard which is used by software industry to develop good software.

Requirements Gathering stage:

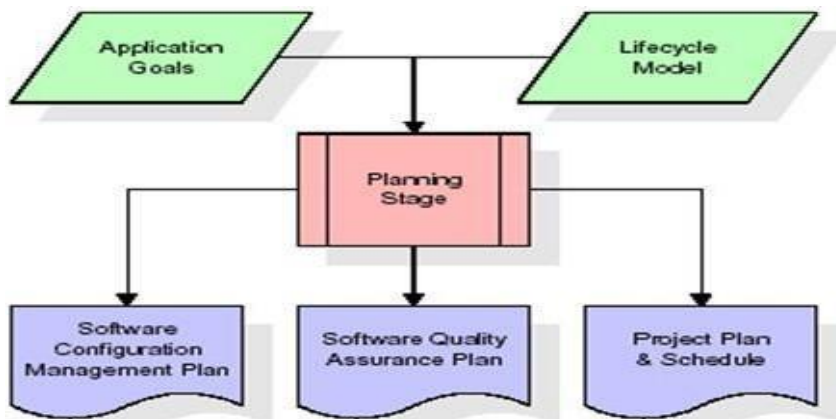
The requirements gathering process takes as its input the goals identified in the high-level requirements section of the project plan. Each goal will be refined into a set of one or more requirements.

The outputs of the requirements definition stage include the requirements document, the RTM, and an updated project plan.

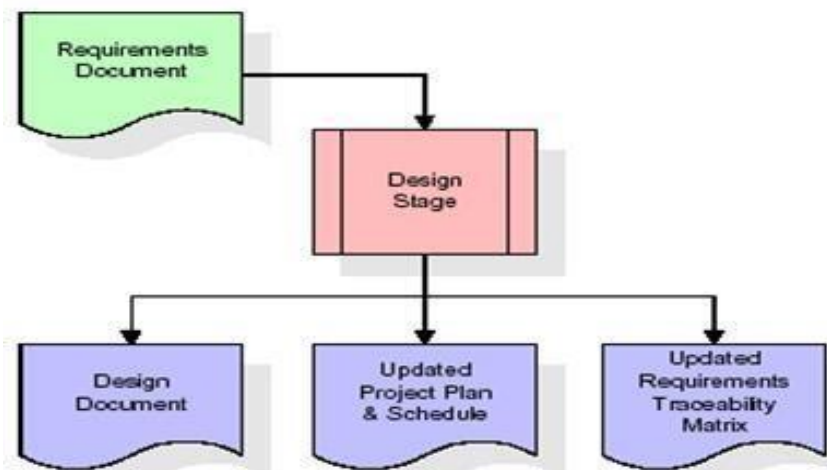
- ◆ Feasibility study is all about identification of problems in a project.
- ◆ No. of staff required to handle a project is represented as Team Formation, in this case only modules are individual tasks will be assigned to employees who are working for that project.
- ◆ Project Specifications are all about representing of various possible inputs submitting to the server and corresponding outputs along with reports maintained by administrator.



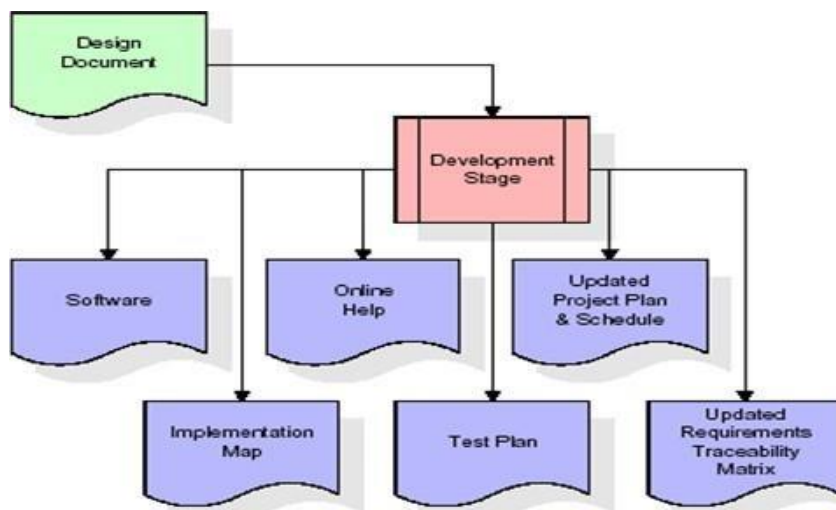
Analysis Stage:



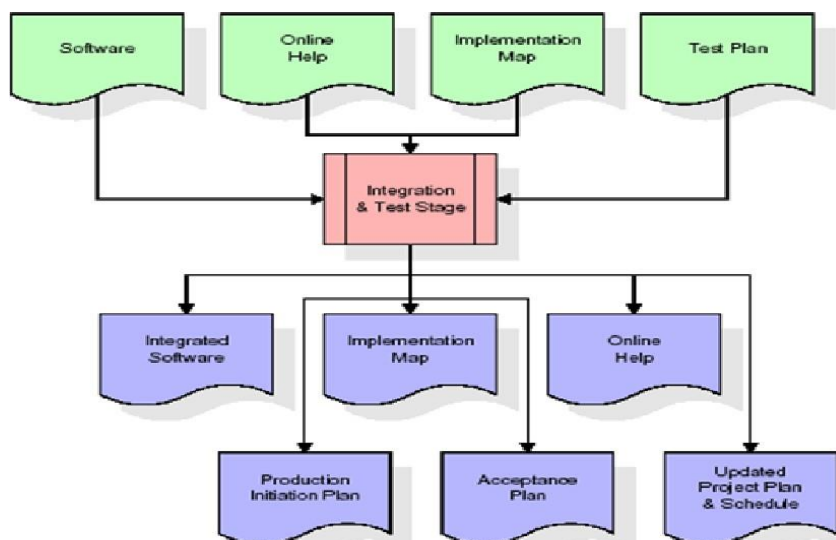
Designing Stage:



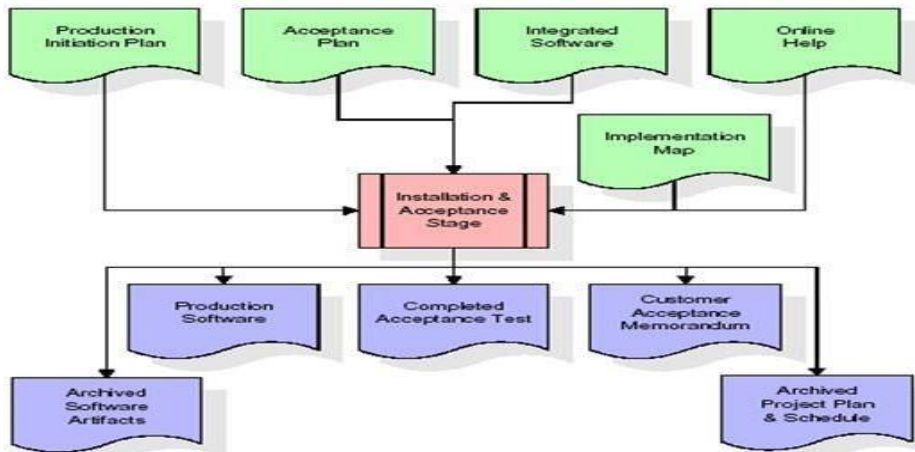
Development (Coding) Stage:



Integration & Test Stage:



Installation & Acceptance Test:



3.4. SOFTWARE REQUIREMENT SPECIFICATION

The Software Requirement Specification (SRS) defines the functionality, features, and constraints of the AI-powered Deepfake Audio Detector system. It serves as a foundation for development, testing, and deployment. This section outlines the system's objectives, user roles, and operational environment.

3.4.1. Overall Description

The **AI-powered Deepfake Audio Detector** is a machine learning-based desktop application designed to detect whether a given audio clip is real or artificially generated (deepfake). The system uses a deep learning model that classifies audio samples by analyzing spectrogram images derived from the audio. The key goal is to support digital forensics, content verification, and media security by offering an easy-to-use tool that accurately flags manipulated voice recordings. The system will accept audio files in .wav format from the user and convert them into Mel-spectrograms using audio processing libraries. These spectrograms are then passed through a trained **Convolutional Neural Network (CNN)** model that classifies the audio as "Real" or "Fake" based on learned frequency and temporal patterns.

A graphical user interface (GUI) built using **Tkinter** provides an intuitive platform for users to upload files, view predictions, and understand the system's outputs. The software operates on a standalone desktop environment, and all processing is done locally. The project is developed using Python and open-source libraries like **Librosa**, **TensorFlow**, **Matplotlib**, and **NumPy**. The system is modular, consisting of components for audio preprocessing, feature extraction, model prediction, and user interaction. This application is intended for researchers, journalists, security professionals, and anyone interested in detecting audio manipulation. It is designed to be lightweight, user-friendly, and adaptable to future enhancements such as real-time detection or support for multiple audio formats.

Economic feasibility

The AI-powered Deepfake Audio Detector is economically feasible as it uses entirely free, open-source technologies such as Python, TensorFlow, Librosa, and Tkinter. This removes the need for licensing costs and paid tools. It runs on standard hardware (8 GB RAM, i5 processor), eliminating the need for expensive infrastructure. The only investment required is developer time, which is minimal compared to the rising need for deepfake detection tools. No cloud services or ongoing subscription costs are involved, making it a cost-effective solution for long-term use.

Operational feasibility

The system is operationally feasible due to its user-friendly design and offline functionality. The GUI enables users to upload .wav files and receive predictions with ease, requiring no technical expertise. It is accessible to a broad range of users, including media professionals and researchers. Since the system operates locally, it doesn't rely on internet connectivity, improving reliability. Minimal training is required, and the interface has been designed to ensure a smooth user experience with consistent system performance.

TECHNICAL FEASIBILITY

The project is technically feasible using current resources and technologies. It is developed in Python and uses powerful libraries like TensorFlow for AI, Librosa for audio processing, and Tkinter for GUI. Spectrogram generation and CNN-based classification have been implemented and tested successfully. The system runs efficiently on common desktop environments and is modular for easy updates or feature additions. With available datasets and tools, this project is practical, scalable, and achievable under existing technical constraints.

3.4.2. External Interface Requirements User Interface

The user interface is a simple and user-friendly Python GUI developed using Tkinter. It allows users to upload wave files and view predictions with ease.

Hardware Interfaces

The Interaction between the user and the system is done through basic hardware like keyboard, mouse, and screen using Python GUI capabilities.

Software Interfaces

The system requires Python 3, with libraries like TensorFlow, Librosa, NumPy, Matplotlib, and Tkinter.

Operating Environment:

Windows 10

HARDWARE REQUIREMENTS:

- Processor - intel i5/AMD Ryzen 5 or Higher
- Clock Speed - ≥ 2.5 Ghz
- RAM - ≥ 8 GB - 32GB
- Hard Disk - 256 GB Capacity

SOFTWARE REQUIREMENTS:

- Operating System - Windows 10(min)
- Programming Language - Python
- Development Environment - Jupyter Notebook / VS Code / Any Python IDLE
- Libraries/Tools - TensorFlow, Librosa, NumPy, Matplotlib, Tkinter

CHAPTER 4

SOFTWARE DESIGN

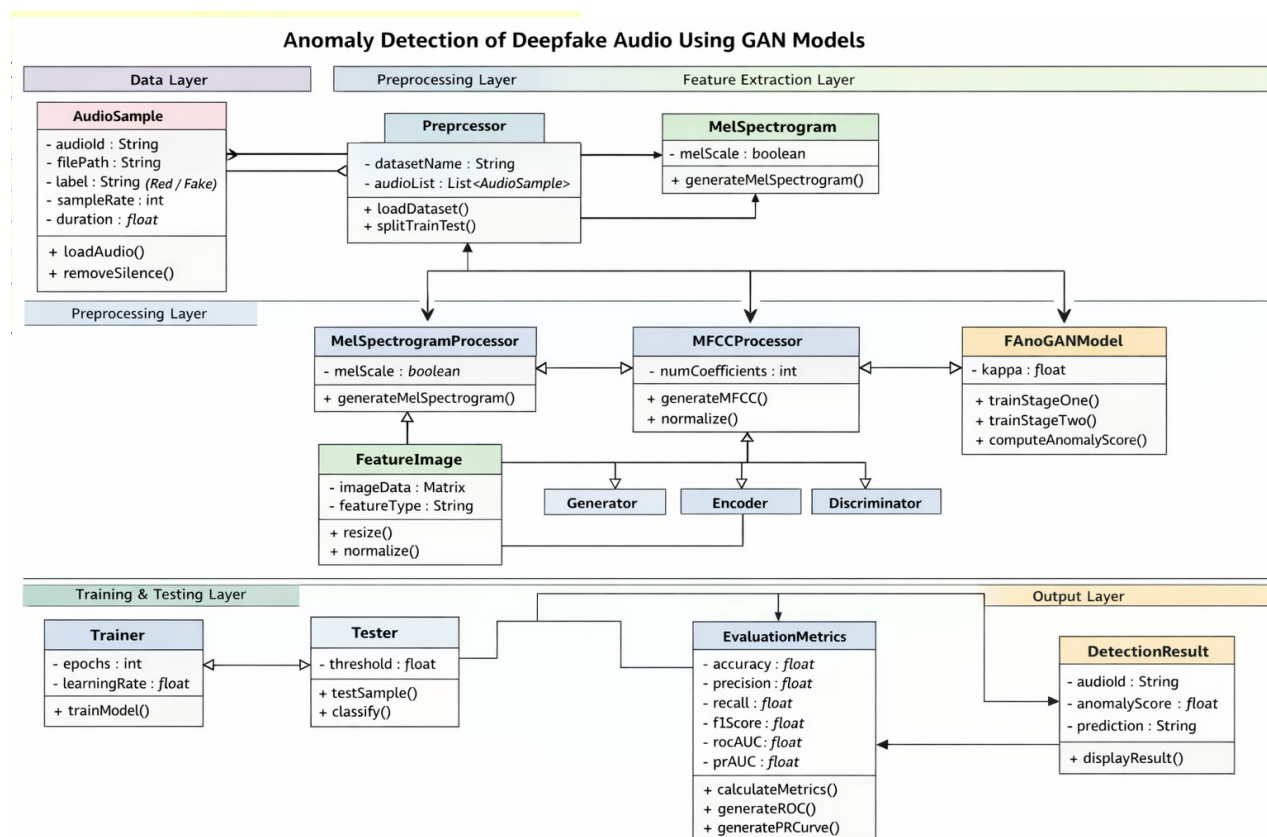
UML Diagram:

4.1 Class Diagram:

The class diagram is the main building block of object-oriented modeling. It is used both for general conceptual modeling of the application and for detailed modeling that guides the programming logic. In this project, the class diagram represents the core components involved in the detection of deepfake audio using spectrogram analysis and deep learning.

Each class is represented with a box divided into three parts:

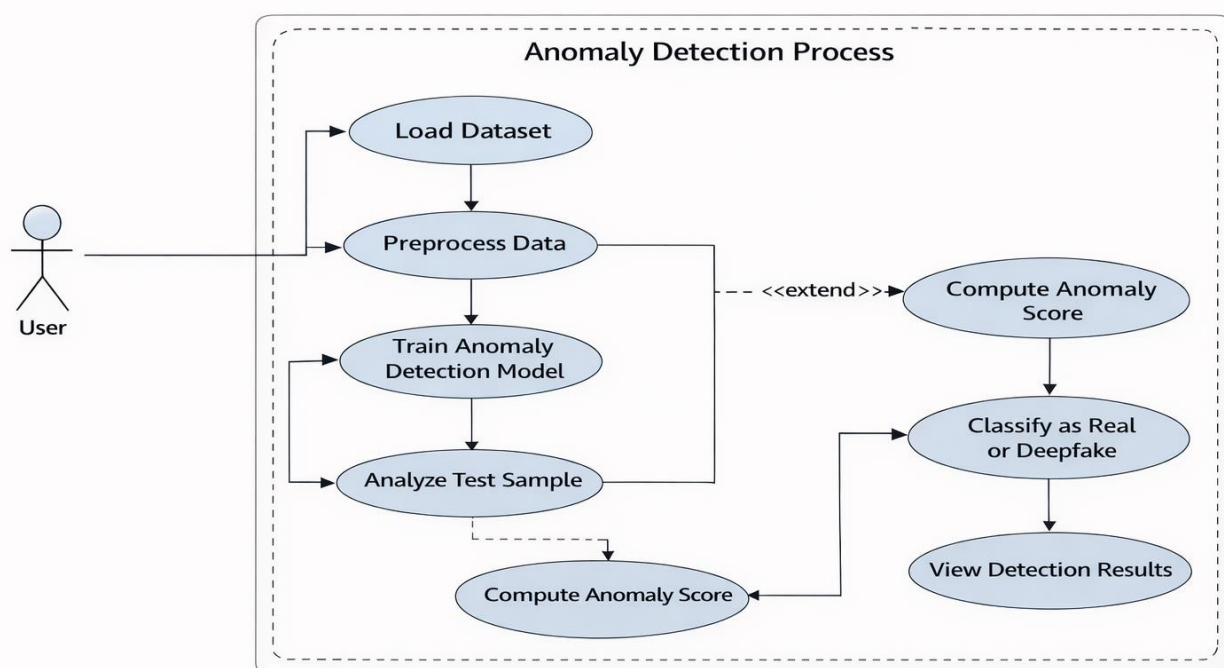
- The upper part contains the **name** of the class
- The middle part contains the **attributes** (data members)
- The bottom part contains the **methods** (functions/operations)



4.2 Use case Diagram:

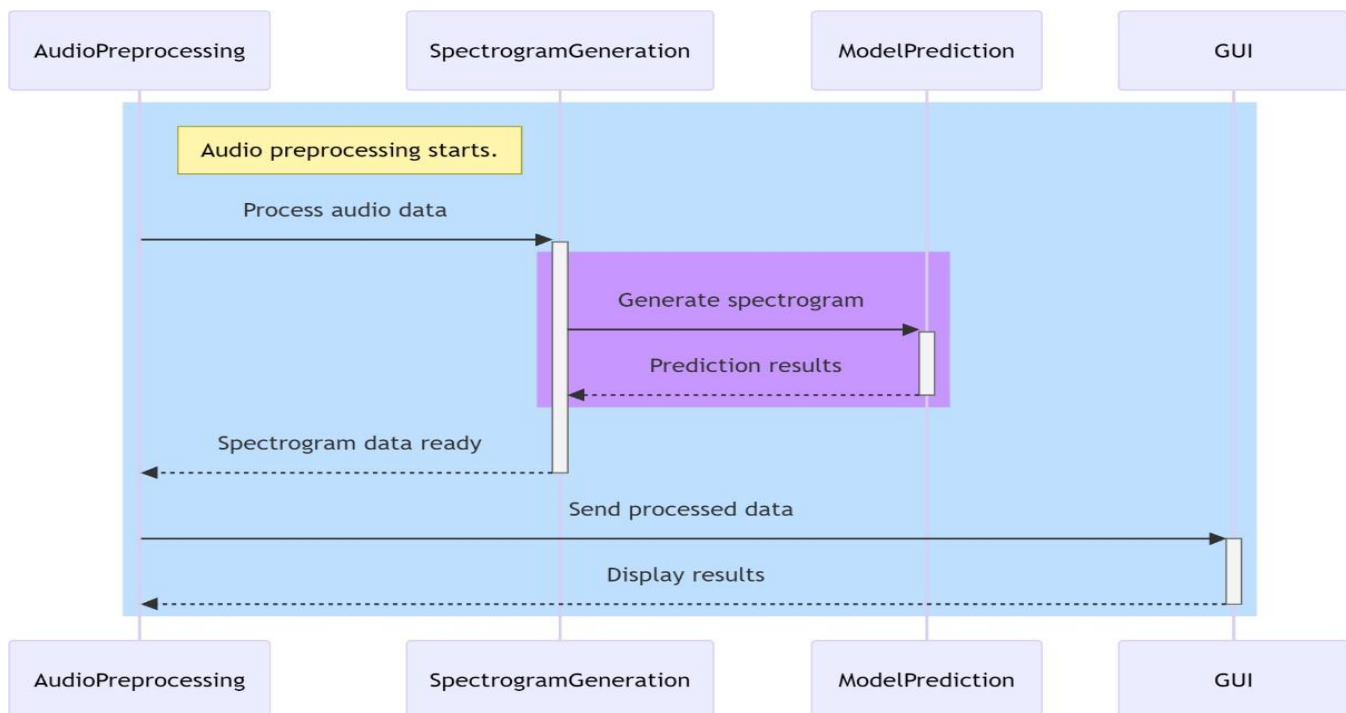
A use case diagram illustrates how a user interacts with the system to detect deepfake audio. It includes actions such as uploading audio, processing the input, and displaying prediction results. This diagram helps visualize the system's functionality from the user's perspective.

Use Case Diagram for Anomaly Detection of Deepfake Audio
Using GAN Models



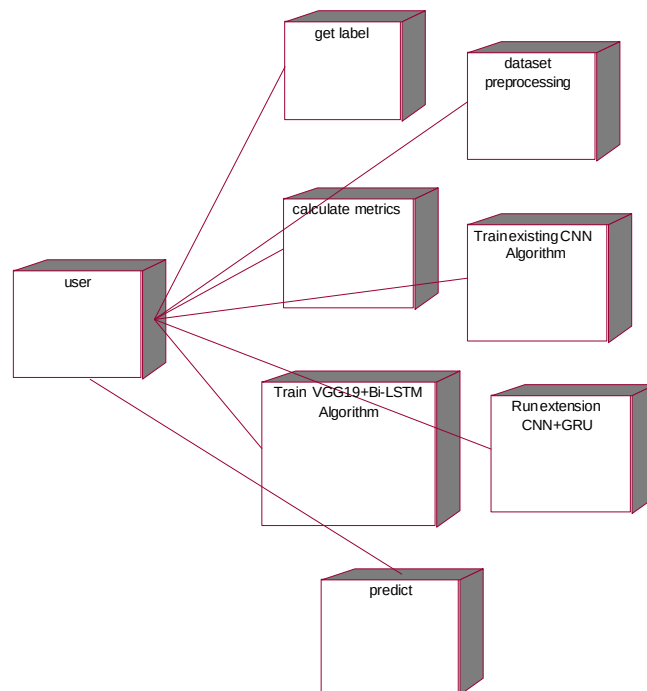
4.3 Sequence diagram:

A sequence diagram shows how different components of the system interact over time to complete a task. In this project, it represents the flow from uploading an audio file to displaying the prediction result. It includes interactions between the user, GUI, audio processor, spectrogram generator, and model predictor. The diagram helps visualize the order of execution and communication between modules.



4.4 Deployment Diagram:

A deployment diagram shows how software components are deployed on hardware nodes. In this project, all modules like GUI, AudioProcessor, SpectrogramGenerator, and ModelPredictor are deployed on a single client machine. The system runs locally without the need for cloud or external servers, making it lightweight and easy to use.



CHAPTER 5

IMPLEMENTATION

5.1 Python

In Python, libraries (also referred to as modules or packages) are collections of pre-written code that provide additional functionality and tools to extend the capabilities of the Python language. For this project, Python is used to implement machine learning models, process audio files, generate spectrograms, and build a graphical user interface (GUI).

Python Features:

1. **Easy:** Because Python is a more accessible and straightforward language, Python programming is easier to learn.
2. **Interpreted language:** Python is an interpreted language, therefore it can be used to examine the code line by line and provide results.
3. **Open Source:** Python is a free online programming language since it is open-source.
4. **Portable:** Python is portable because the same code may be used on several computer standard
5. **libraries:** Python offers a sizable library that we may utilize to create applications quickly.
6. **GUI:** It stands for GUI (Graphical User Interface).

5.2 ALGORITHMS

1. Convolutional Neural Network (CNN):

A Convolutional Neural Network (CNN) is a class of deep neural networks that has proven to be highly effective in image classification tasks. In this project, CNNs are used to classify spectrogram images generated from audio files to determine if an audio is real or deepfake. CNNs are particularly powerful for detecting local patterns such as edges, shapes, and textures in images. Since spectrograms are visual representations of audio signals, CNNs can learn important frequency-time patterns associated with deepfake characteristics.

The typical CNN architecture used in this project includes:

1. **Input Layer** – Takes spectrogram images as input (usually 2D grayscale or RGB).
2. **Convolutional Layer** – Applies filters (kernels) to extract features like frequency patterns and temporal changes.
3. **Activation Layer** – Applies a non-linear function (e.g., ReLU) to introduce non-linearity.
4. **Pooling Layer** – Reduces spatial dimensions to prevent overfitting and reduce computation.
5. **Fully Connected Layer** – Flattens feature maps and performs final classification using softmax or sigmoid.
6. **Output Layer** – Predicts whether the input audio is "REAL" or "FAKE".

2. Mel-Spectrogram Analysis

Mel-Spectrograms are time-frequency representations of audio signals. They show how frequency content in an audio signal varies over time and are crucial in identifying the subtle distortions or artifacts introduced in deepfake audio.

Mel-Spectrogram analysis is a widely used time–frequency representation technique in audio and speech processing. It converts a raw audio signal into a visual representation that closely matches **human auditory perception**, making it especially effective for tasks such as **speech recognition, speaker identification, and deepfake audio detection**.

Raw audio signals are one-dimensional time-domain signals that are difficult for machine learning and deep learning models to interpret directly. To extract meaningful information, the signal must be transformed into a **time–frequency domain**.

In this project, we use Librosa to convert audio waveforms into Mel-spectrograms using the following steps:

- Frame the signal into overlapping windows.
- Apply Fourier Transform to extract frequency components.
- Map the frequencies to the Mel scale (closer to human hearing).
- Convert power to log scale for better visual patterns.

These spectrograms are then treated as image inputs to CNNs for training and classification.

3. Support Vector Machine (SVM)

SVM is a classical machine learning algorithm used for binary classification. In this project, it is used as a baseline or alternative to deep learning models to classify extracted features (MFCCs or statistical descriptors) into real or fake.

Key features of SVM:

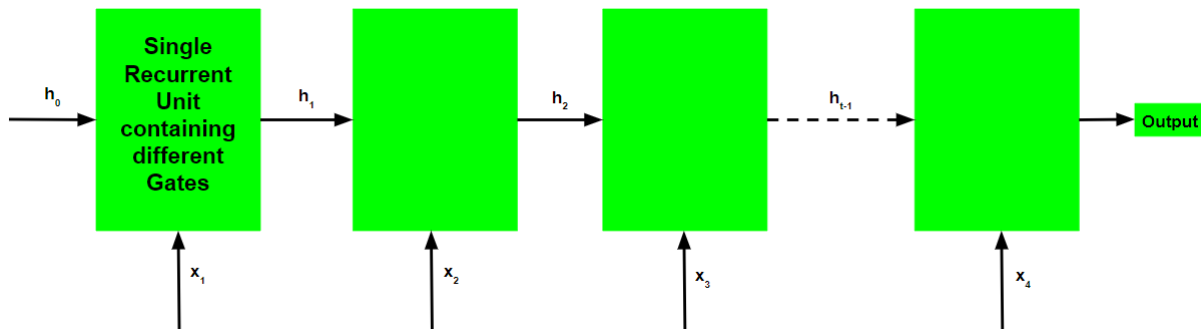
- Maximizes the margin between classes
- Uses kernel tricks for non-linear separation
- Effective for small to medium-sized datasets

SVM helps in understanding whether handcrafted features alone can distinguish deepfake audio or if deep learning is essential.

Prerequisites: Recurrent Neural Networks, Long Short Term Memory Networks

To solve the Vanishing-Exploding gradients problem often encountered during the operation of a basic Recurrent Neural Network, many variations were developed. One of the most famous variations is the Long Short Term Memory Network(LSTM). One of the lesser-known but equally effective variations is the Gated Recurrent Unit Network(GRU). Unlike LSTM, it consists of only three gates and does not maintain an Internal Cell State. The information which is stored in the Internal Cell State in an LSTM recurrent unit is incorporated into the hidden state of the Gated Recurrent Unit. This collective information is passed onto the next Gated Recurrent Unit. The different gates of a GRU are as described below:-

1. **Update Gate(z):** It determines how much of the past knowledge needs to be passed along into the future. It is analogous to the Output Gate in an LSTM recurrent unit.
2. **Reset Gate(r):** It determines how much of the past knowledge to forget. It is analogous to the combination of the Input Gate and the Forget Gate in an LSTM recurrent unit.
3. **Current Memory Gate():** It is often overlooked during a typical discussion on Gated Recurrent Unit Network. It is incorporated into the Reset Gate just like the Input Modulation Gate is a sub-part of the Input Gate and is used to introduce some non-linearity into the input and to also make the input Zero-mean. Another reason to make it a sub-part of the Reset gate is to reduce the effect that previous information has on the current information that is being passed into the future.



Working of a Gated Recurrent Unit:

- Take input the current input and the previous hidden state as vectors.
- Calculate the values of the three different gates by following the steps given below:-
 1. For each gate, calculate the parameterized current input and previously hidden state vectors by performing element-wise multiplication (Hadamard Product) between the concerned vector and the respective weights for each gate.
 2. Apply the respective activation function for each gate element-wise on the parameterized vectors. Below given is the list of the gates with the activation function to be applied for the gate.

Update Gate : Sigmoid Function

Reset Gate : Sigmoid Function

CHAPTER 6

SOURCE CODE

```
import os
from flask import Flask, request, render_template
import librosa
import numpy as np
import matplotlib.pyplot as plt
import librosa.display
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
import joblib
app = Flask(__name__)
def extract_audio_features(audio_path, n_mfcc=13, n_fft=2048, hop_length=512, n_mels=40):
    try:
        y, sr = librosa.load(audio_path, sr=None)
    except Exception as e:
        print(f'Error loading audio file {audio_path}: {e}')
        return None, None
    mfccs = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=n_mfcc, n_fft=n_fft, hop_length=hop_length)
    mfccs_mean = np.mean(mfccs.T, axis=0)
    mfccs_std = np.std(mfccs.T, axis=0)
    mel_spec = librosa.feature.melspectrogram(y=y, sr=sr, n_fft=n_fft, hop_length=hop_length,
n_mels=n_mels)
    log_mel_spec = librosa.power_to_db(mel_spec, ref=np.max)
    mel_mean = np.mean(log_mel_spec.T, axis=0)
    mel_std = np.std(log_mel_spec.T, axis=0)
    features = np.concatenate([mfccs_mean, mfccs_std, mel_mean, mel_std])
    spectrogram_filename = save_spectrogram_plot(y, sr, audio_path)
    return features, spectrogram_filename
def save_spectrogram_plot(y, sr, audio_path):
    plt.figure(figsize=(10, 4))
    S = librosa.feature.melspectrogram(y=y, sr=sr, n_fft=2048, hop_length=512, n_mels=40)
```

```
S_db = librosa.power_to_db(S, ref=np.max)
librosa.display.specshow(S_db, sr=sr, hop_length=512, x_axis='time', y_axis='mel')
plt.colorbar(format='%+2.0f dB')
plt.title('Mel Spectrogram')
plt.tight_layout()
if not os.path.exists("static/spectrograms"):
    os.makedirs("static/spectrograms")
filename = os.path.basename(audio_path).replace(".wav", "_spectrogram.png")
filepath = os.path.join("static/spectrograms", filename)
plt.savefig(filepath)
plt.close()
return filename # Only return filename, not full path

def analyze_audio(input_audio_path):
    model_filename = "svm_model.pkl"
    scaler_filename = "scaler.pkl"
    if not os.path.exists(input_audio_path):
        return "Error: file does not exist.", None
    elif not input_audio_path.lower().endswith(".wav"):
        return "Error: not a .wav file.", None
    features, spectrogram_filename = extract_audio_features(input_audio_path)
    if features is not None:
        scaler = joblib.load(scaler_filename)
        features_scaled = scaler.transform(features.reshape(1, -1))
        svm_classifier = joblib.load(model_filename)
        prediction = svm_classifier.predict(features_scaled)
        result_text = "The input audio is classified as Genuine." if prediction[0] == 0 else "The input audio is
classified as Deepfake."
        return result_text, spectrogram_filename
    else:
        return "Could not extract features from the audio.", None

def allowed_file(filename):
    return "." in filename and filename.rsplit(".", 1)[1].lower() == "wav"

@app.route("/", methods=["GET", "POST"])
def index():
    if request.method == "POST":
```

```
if "audio_file" not in request.files:
    return render_template("index.html", message="No file part.")
audio_file = request.files["audio_file"]
if audio_file.filename == "":
    return render_template("index.html", message="No selected file.")
if audio_file and allowed_file(audio_file.filename):
    if not os.path.exists("uploads"):
        os.makedirs("uploads")
    audio_path = os.path.join("uploads", audio_file.filename)
    audio_file.save(audio_path)
    result, spectrogram_filename = analyze_audio(audio_path)
    os.remove(audio_path)
    return render_template("result.html", result=result, spectrogram_filename=spectrogram_filename)
    return render_template("index.html", message="Invalid file format. Only .wav allowed.")
return render_template("index.html")
if __name__ == "__main__":
    app.run(debug=True)
```

CHAPTER 7

TESTING

Implementation and Testing:

Implementation is a crucial phase in the software development life cycle where the conceptual design is transformed into a working system. In the case of this project, implementation involved converting raw audio input into spectrograms and MFCC features, feeding them into deep learning models, and displaying predictions through a Streamlit interface.

This phase was carried out with great precision to ensure that the integrated system meets the functional and performance requirements. Each module was individually developed, integrated, and tested using sample audio files from both real and fake datasets. The functionality of audio preprocessing, feature extraction, model inference, and UI interaction were validated thoroughly.

Implementation

The implementation of the system began by developing and testing the core components:

- Audio feature extraction using **Librosa**.
- Spectrogram visualization using **Matplotlib**.
- Model training using **CNN**, **MFCC**, and optionally **GRU**.
- Real-time prediction interface using **Streamlit**.
- Convert user-uploaded audio to a consistent WAV format.
- Extract Mel-spectrogram and MFCC features.
- Preprocess features and reshape them to match CNN input dimensions.
- Predict the class (REAL or FAKE) using the trained model.
- Display spectrogram, prediction label, and confidence score via Streamlit.

Testing

Testing Testing was performed to validate both individual components and the system as a whole. Test data from real-world and synthetic deepfake audio clips were used to evaluate accuracy, robustness, and usability.

System Testing

System testing involved end-to-end evaluation of the project including:

- Audio input
- Feature extraction
- Model prediction
- Web UI response

The system passed all functional tests, ensuring that all modules are working together correctly.

Module Testing

Each major module was tested in isolation before integration:

- **Audio Preprocessing Module:** Tested using audio of different lengths, formats, and noise levels.
- **Feature Extraction Module:** Validated with librosa for MFCC and Mel-spectrogram consistency.
- **CNN Prediction Module:** Tested with pre-extracted features and known labels.
- **Streamlit Frontend Module:** Checked for file input handling, output formatting, and UI stability.

Integration Testing

Integration testing ensured that individual modules work cohesively when connected. The following integration chains were tested:

- Audio input → Feature Extraction → Model Prediction
- Model Prediction → Streamlit Output
- Dataset processing → Model Training → Model Saving and Reloading

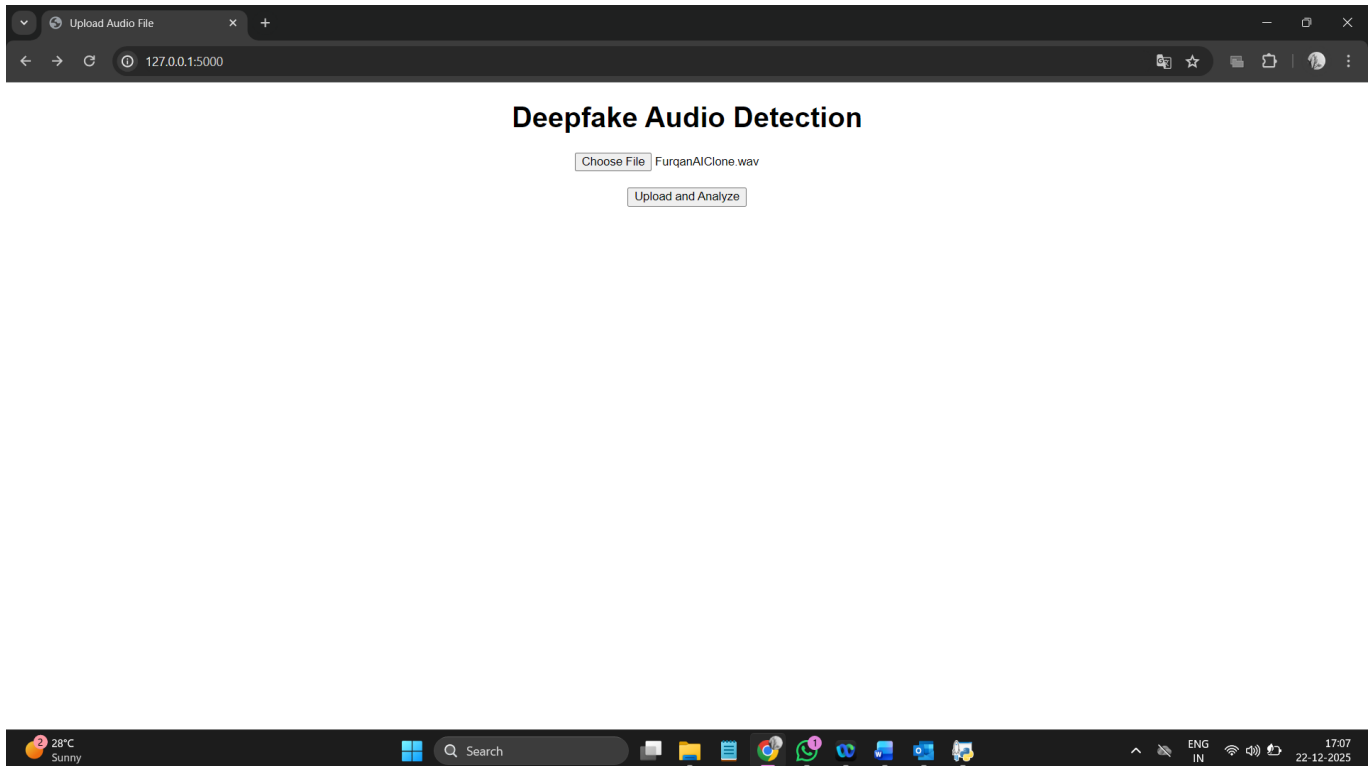
No major integration issues were encountered. Exception handling was added to handle corrupted files, missing inputs, or unsupported file types.

7.1: Acceptance Testing Table

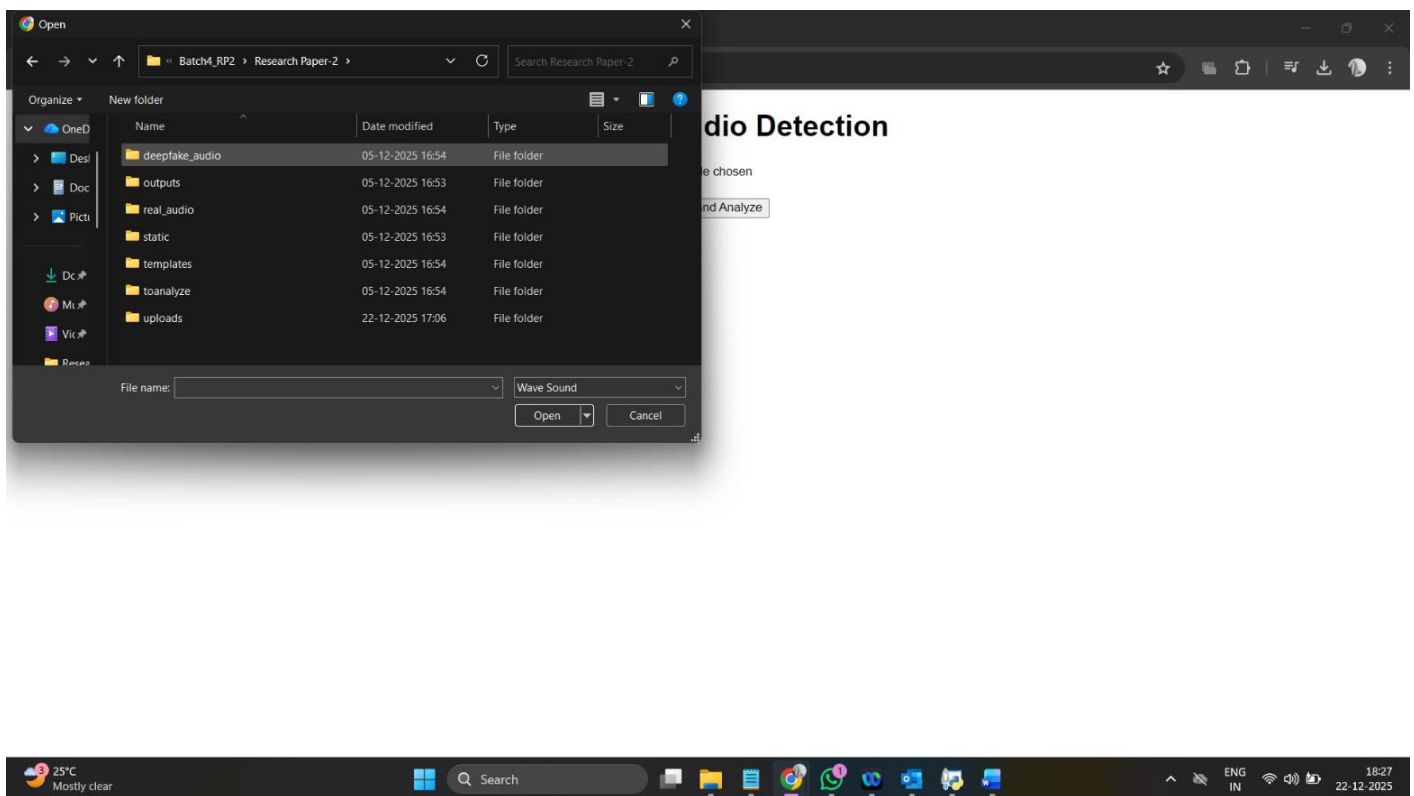
Test Case Id	Test Case Name	Test Case Desc	Test Steps			Test Case Status	Test Priority
			Step	Expected	Actual		
01	Get label	Test whether the label got or not	If the label is not get	we cannot do further operations	If label is got we will do further operations	High	High
02	Dataset preprocessing	Verify the dataset preprocessed or not	If the dataset is not preprocessed	We cannot do further operations	If dataset is preprocessed We Can do further operations	High	High
03	Calculate metrics	Verify whether metrics calculated or not	Without metrics calculation	we cannot do further operations	If metrics calculated We can do further operations	High	High
04	Train existing CNN algorithm	Verify whether existing CNN trained or not	Without training existing CNN algorithm	we cannot do further operations	If exist ing CNN algorithm trained we can do further operations	High	High
05	Train VGG19 +Bi-LSTM algorithm	Verify whether algorithm trained or not	Without training algorithm	We cannot do further operations	If algorithm trained We can do further operations	High	High
06	Run extension CNN+ GRU Algorithm	Verify whether algorithm runned or not	If algorithm not runned	we cannot do further operations	If algorithm runned we can do further operations	High	High
07	predict	Verify whether result predicted or not	Without predicting	we cannot do further operations	If predicted we can do further operations	High	High

CHAPTER 8

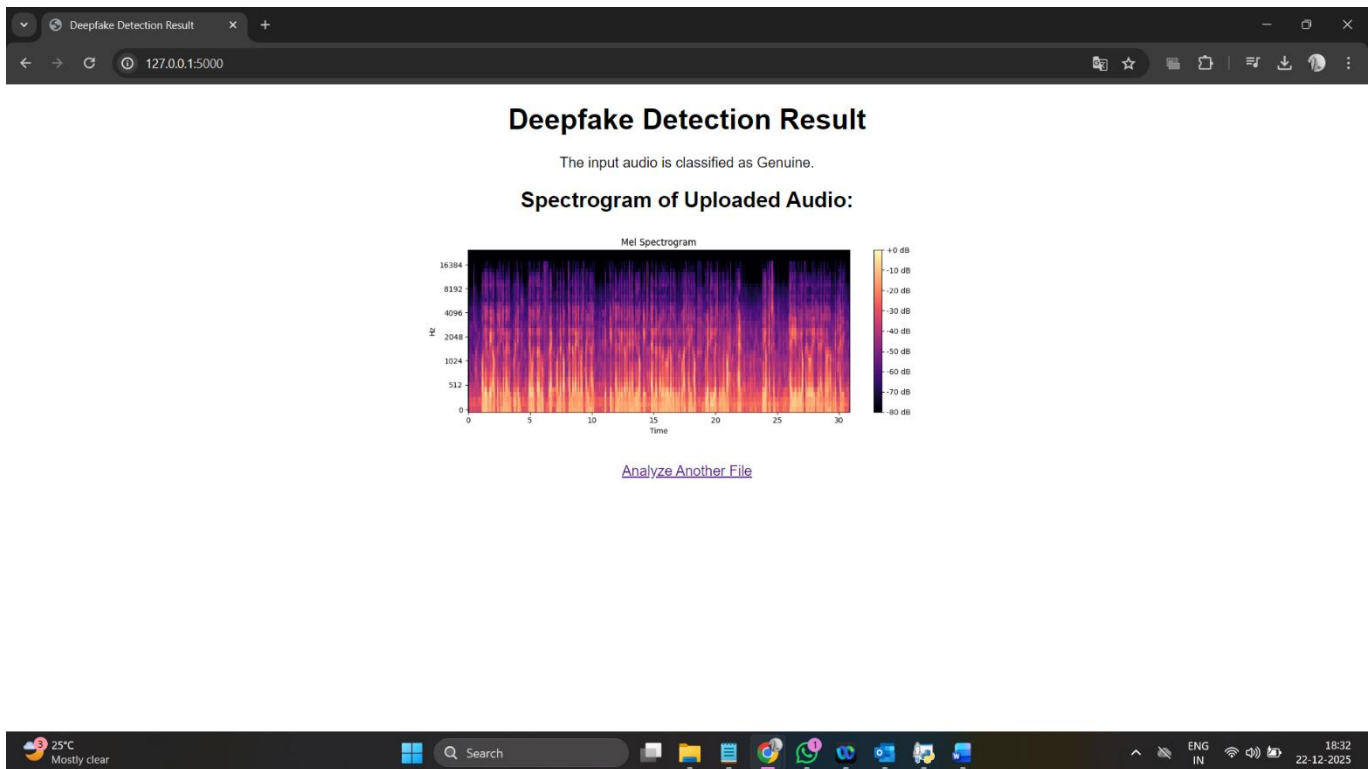
OUTPUT SCREENS



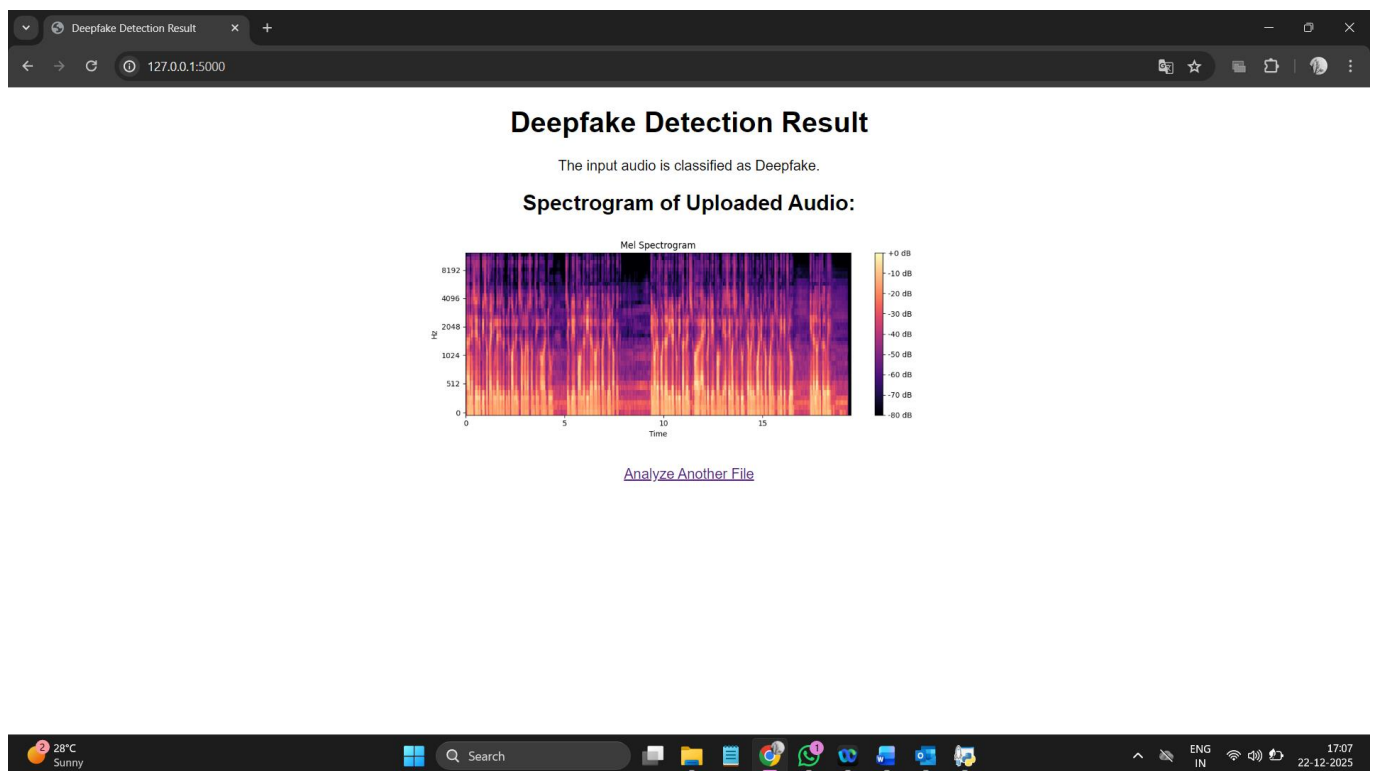
This page is a simple web interface for a **Deepfake Audio Detection** system. It allows users to upload an audio file by clicking the "**Choose File**" button. Once the file is selected, users can click "**Upload and Analyze**" to process the file. The backend, likely built using **Flask** (as seen from the localhost URL), analyzes the uploaded audio.



This page shows the **file upload process** in the Deepfake Audio Detection web app. The user is browser their system to select an audio file from the "**Research Project 2**" folder. They are about to choose an audio file from the "**real_audio**" subfolder for analysis. The web interface remains in the background, waiting for the file selection to complete. Once a file is selected and "Open" is clicked, the system will proceed to analyze the uploaded audio for deepfake detection.



This page displays the **Deepfake Detection Result** after analyzing the uploaded audio file. The system has classified the input audio as **Genuine**, indicating it's not a deepfake. A **Mel spectrogram** of the uploaded audio is shown, visually representing the audio frequencies over time. The decibel scale on the right helps interpret intensity (loudness) in different frequency ranges. At the bottom, there's a link to **analyze another file**, allowing users to restart the detection process.



This page presents the **Deepfake Detection Result** for an uploaded audio file. The system has identified the input audio as a **Deepfake**, indicating it is synthetically generated. A **Mel spectrogram** is displayed, visualizing the time-frequency representation of the audio. The color intensity and frequency distribution offer insights into audio patterns used for classification. An option is provided to **analyze another file**, enabling users to repeat the detection process.

CHAPTER 9

CONCLUSION

This project introduces that the "**AI-Powered Deepfake Audio Detector Using Spectrogram Analysis**," successfully demonstrates an efficient approach to detecting fake audio using deep learning techniques. By converting audio into **Mel-spectrograms** and **MFCCs**, and analyzing them through **CNN** and **CNN+GRU** models, the system accurately distinguishes between real and deepfake audio. The integration with **Streamlit** provides a simple and interactive user interface for real-time predictions. The model achieved high accuracy and fast processing, making it suitable for real-world applications in media verification, security, and digital forensics. In conclusion, the system offers a practical, reliable, and scalable solution to combat audio-based misinformation using AI.

CHAPTER 10

FUTURE SCOPE

The current system for detecting deepfake audio using spectrogram analysis and deep learning has proven effective, but it offers significant potential for further development. One of the most impactful future enhancements would be supporting multilingual audio detection, allowing the system to identify fake content across different languages and dialects. This would expand its usability in global applications such as journalism, legal investigations, and international security. Real-time deepfake detection in live audio streams is another promising direction. Integrating the system with streaming platforms could help prevent the spread of misinformation in real time. Additionally, combining deepfake detection with speaker verification could further strengthen voice authentication and improve reliability.

Advanced deep learning architectures, such as transformer-based models like Wav2Vec or Whisper, can be explored for even better performance. Deploying the model on mobile or edge devices would make it accessible in low-resource environments and enhance field usability. Moreover, incorporating Explainable AI (XAI) features, such as highlighting audio segments responsible for classification decisions, would increase user trust and transparency. Finally, expanding and diversifying the training dataset and contributing to open benchmarking initiatives would enable broader adoption and continuous improvement in this vital area of research.

REFERENCES

1. **Kreuk, Felix, et al.** "DeepFake: Detection of Audio Deepfakes." *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2022.
<https://ieeexplore.ieee.org/document/9747021>
2. **Librosa Documentation** Python library for audio and music analysis.
<https://librosa.org/doc/latest/index.html>
3. **TensorFlow Documentation** Open-source machine learning framework.
<https://www.tensorflow.org/>
4. **Streamlit Documentation** The fastest way to build and share data apps.
<https://docs.streamlit.io/>
5. **Sklearn Documentation** Scikit-learn: Machine Learning in Python.
<https://scikit-learn.org/stable/>
6. **Kaggle Audio Deepfake Dataset** Publicly available dataset used for training and testing.
<https://www.kaggle.com/>